

Deep learning for medical imaging school

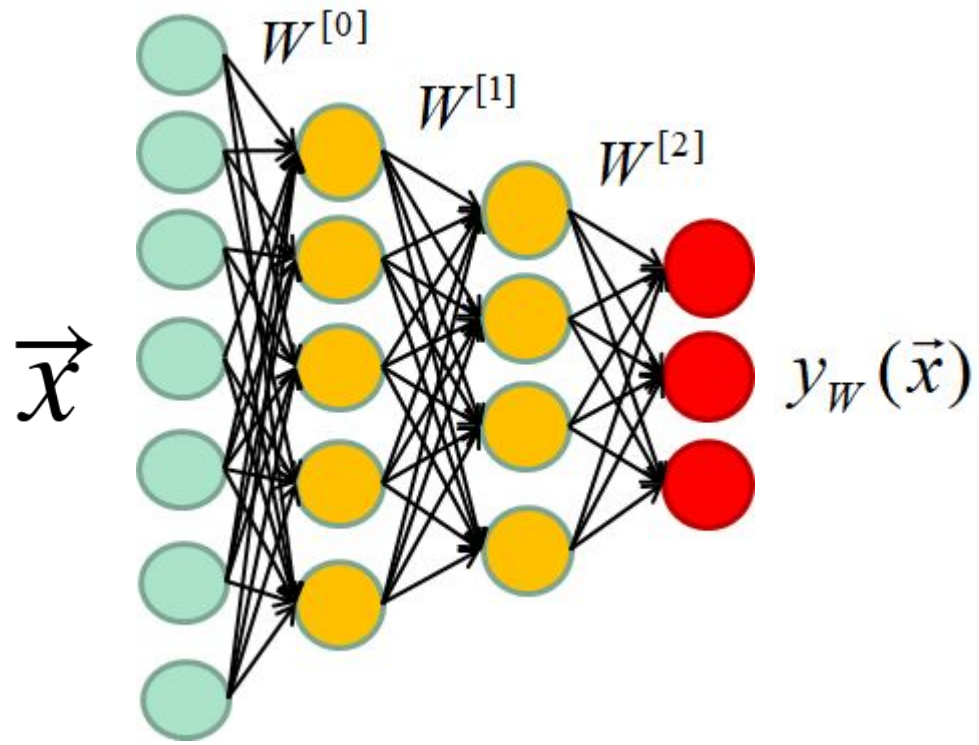
Hands-on session

Autoencoders

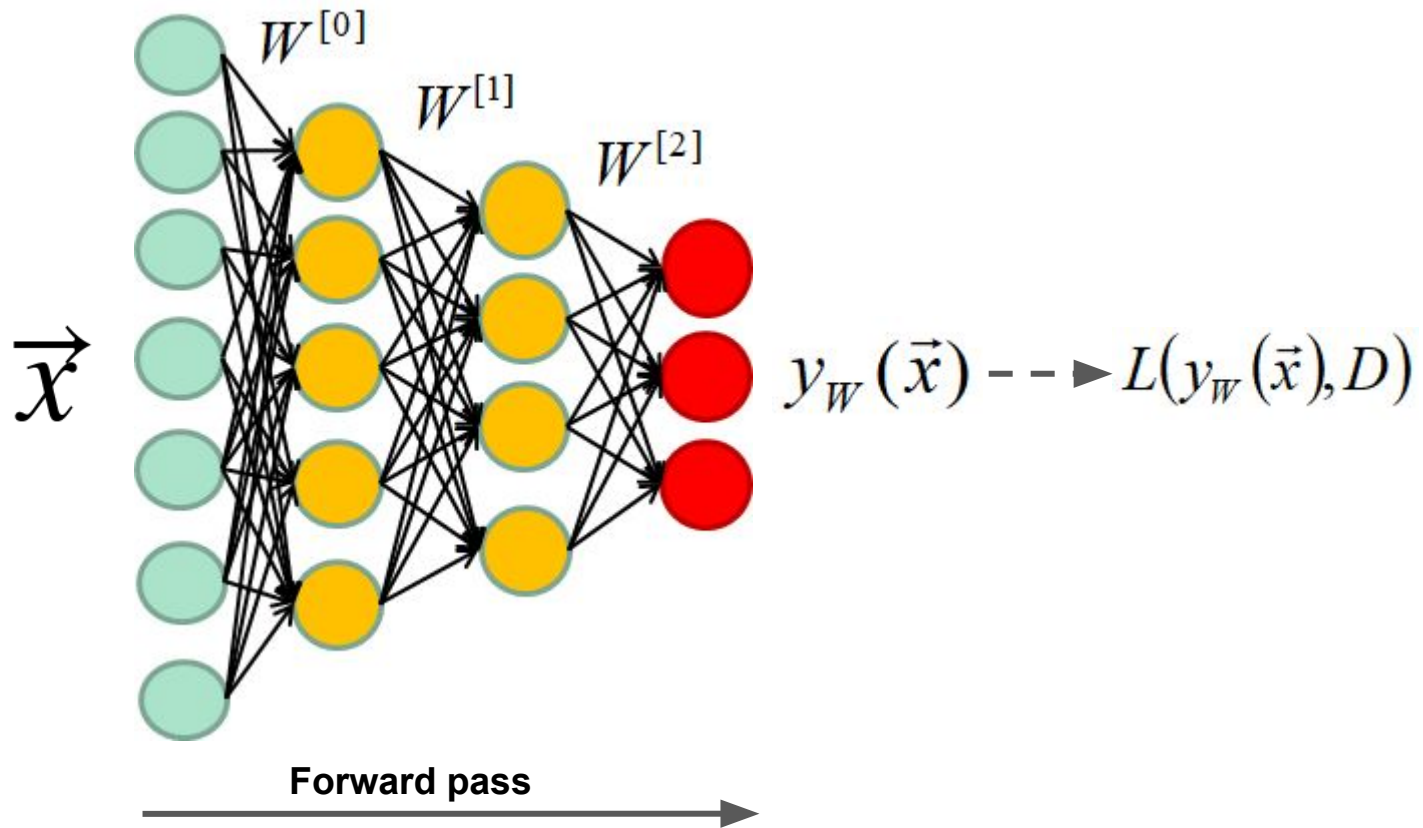
By

Pierre-Marc Jodoin, Thierry Judge

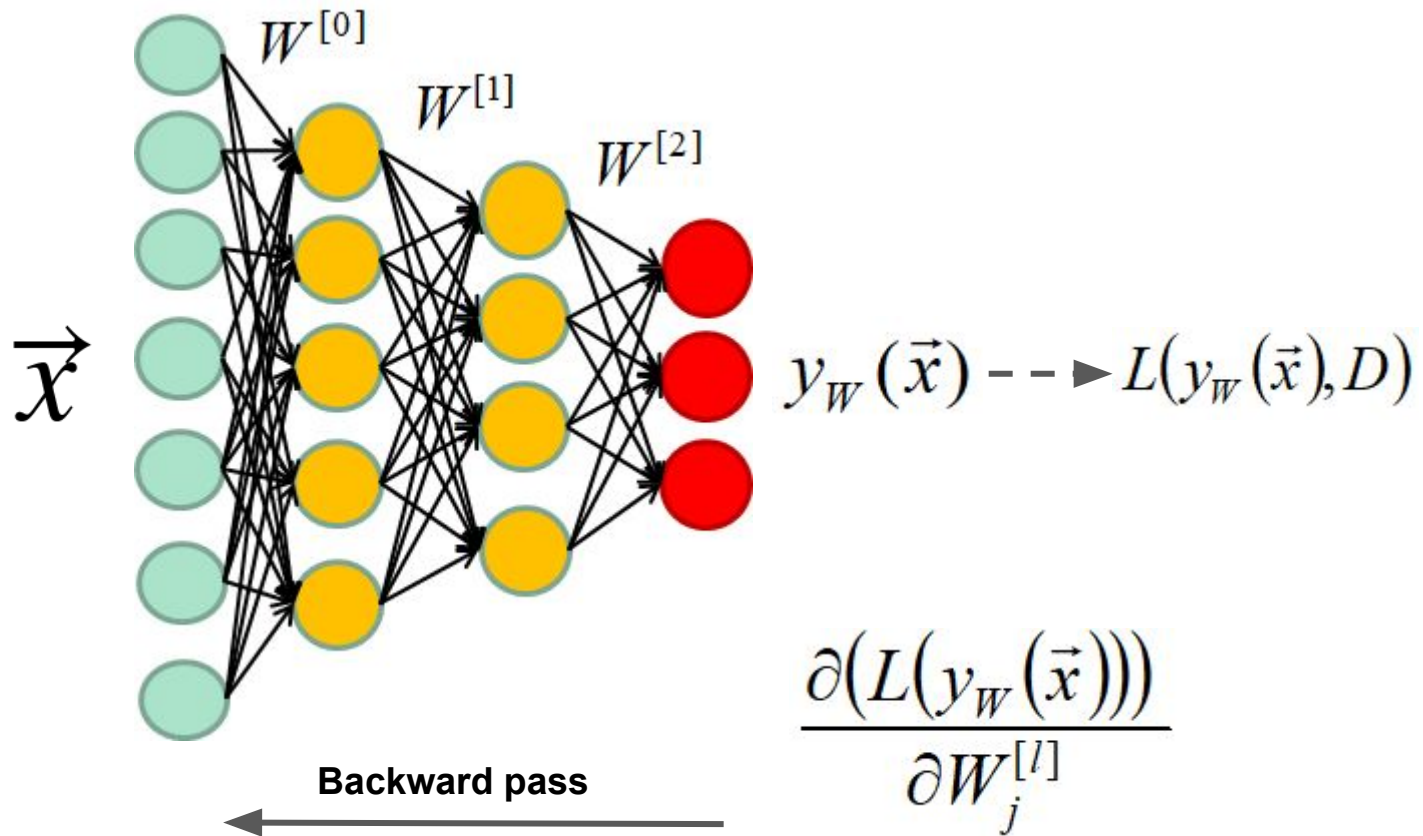




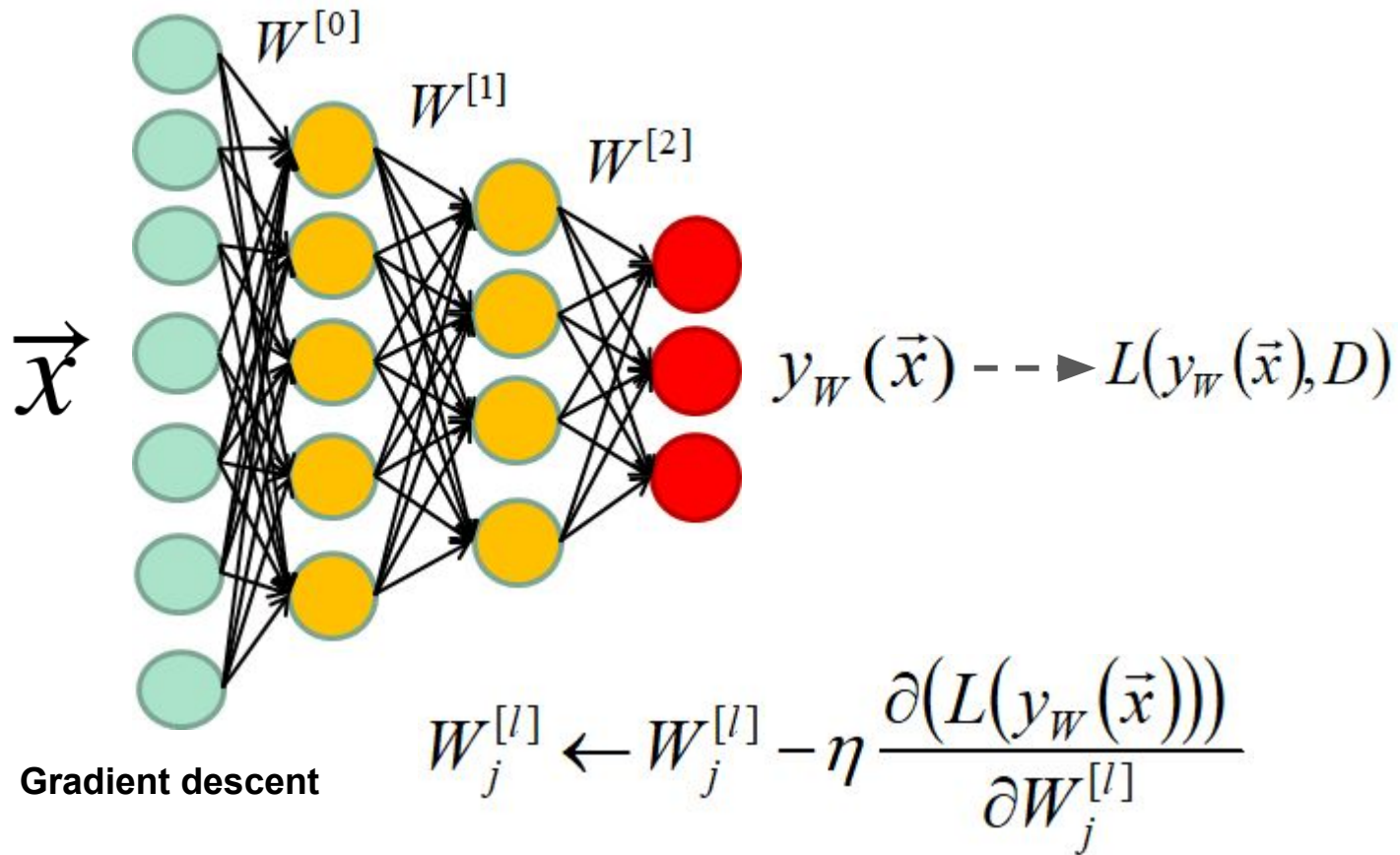
Annotated dataset: $D = \{(\vec{x}_1, t_1), (\vec{x}_2, t_2), \dots, (\vec{x}_N, t_N)\}$

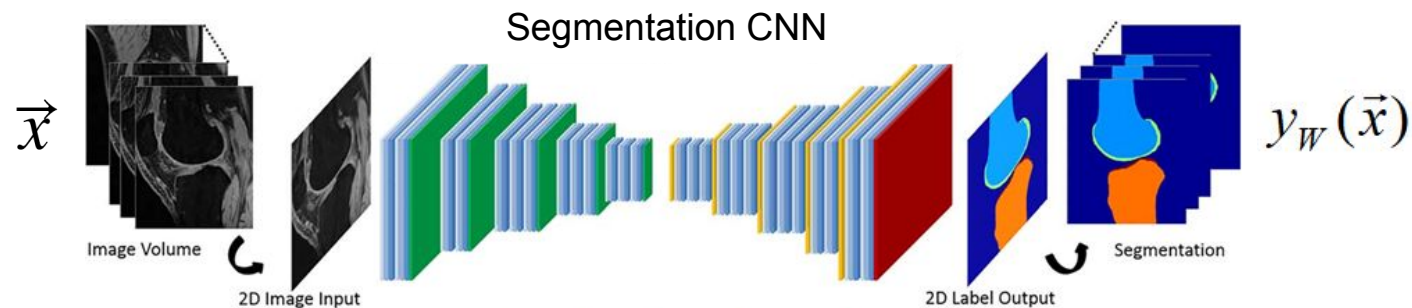
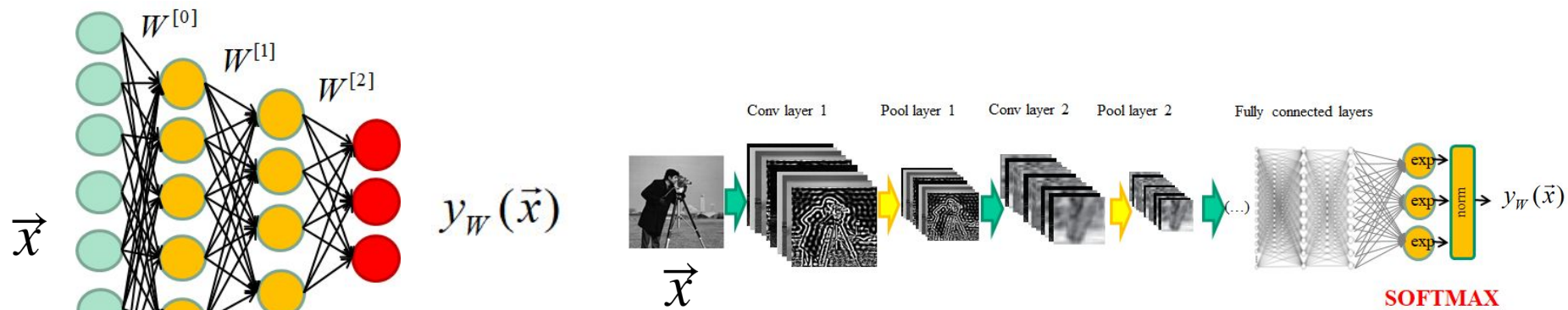


Annotated dataset: $D = \{(\vec{x}_1, t_1), (\vec{x}_2, t_2), \dots, (\vec{x}_N, t_N)\}$



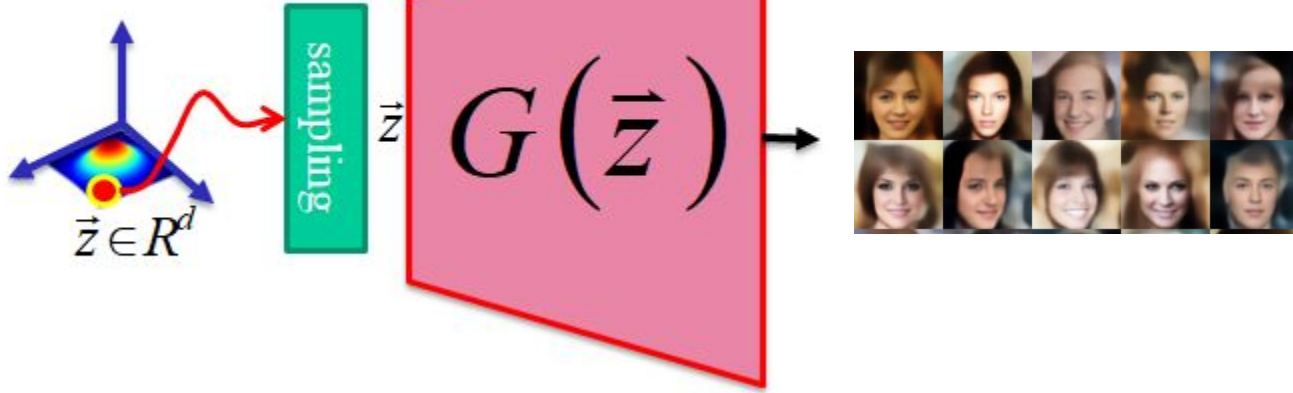
Annotated dataset: $D = \{(\vec{x}_1, t_1), (\vec{x}_2, t_2), \dots, (\vec{x}_N, t_N)\}$





GAN

(Latent space)

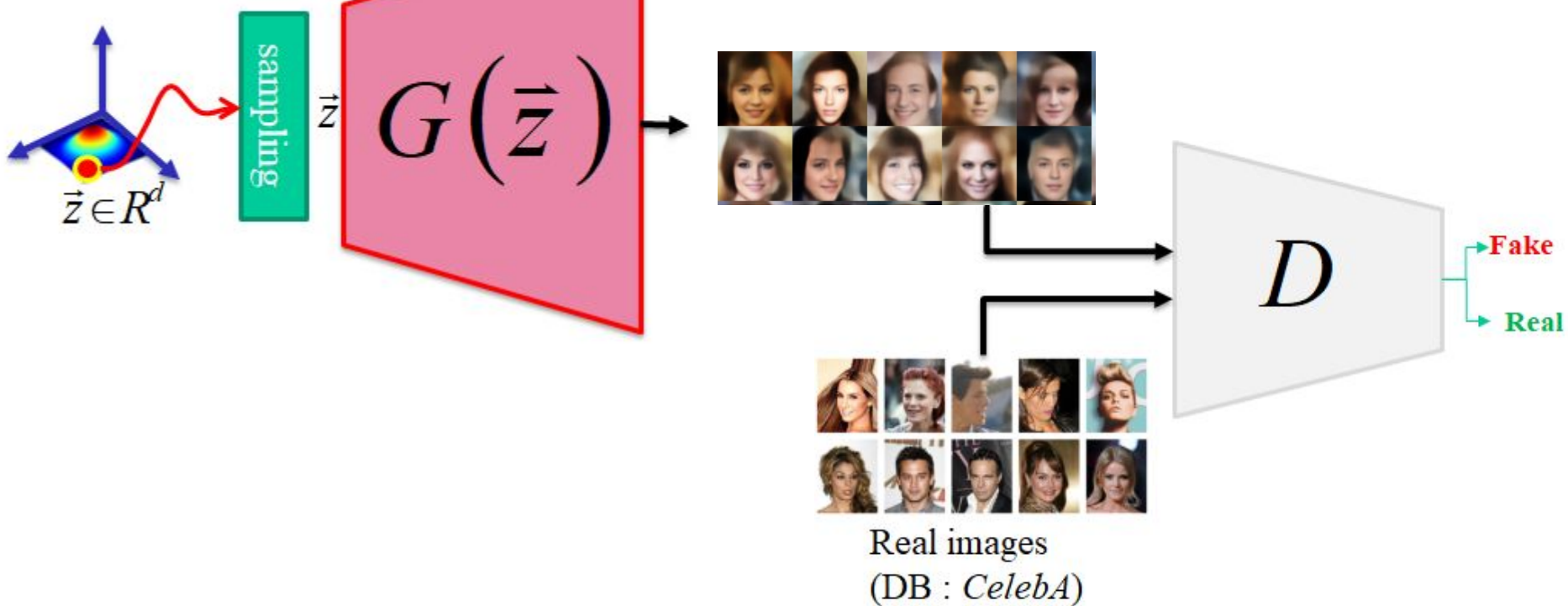


Loss?

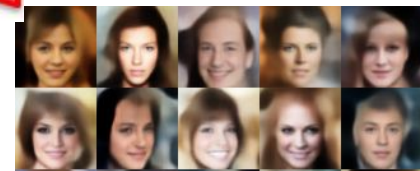
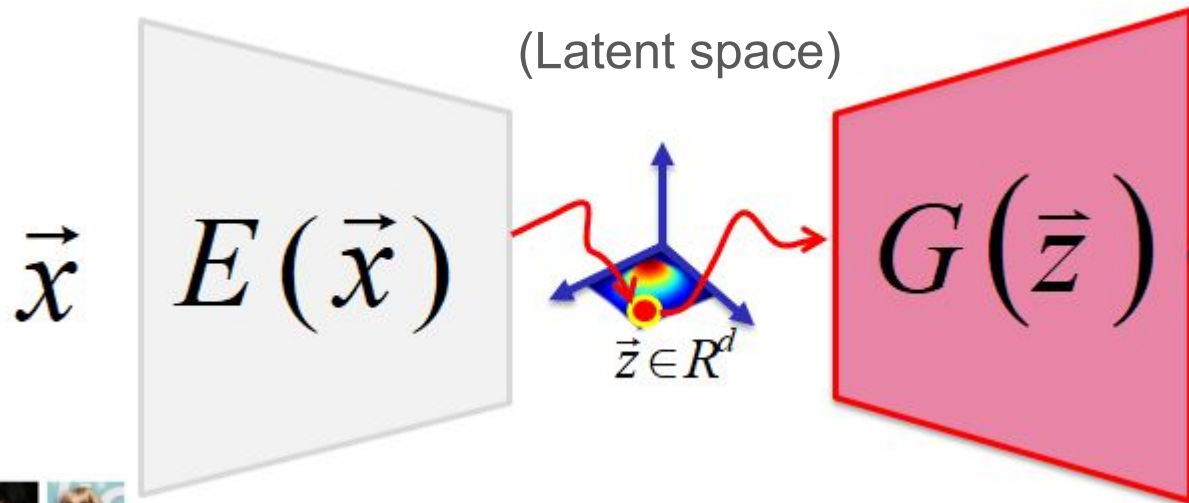
Annotated dataset?

GAN

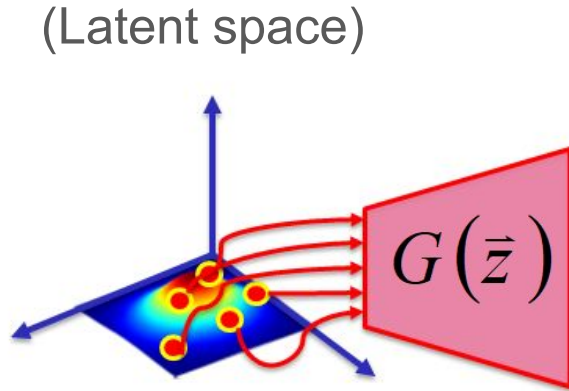
(Latent space)



Autoencoders



Autoencoders (once training is over)



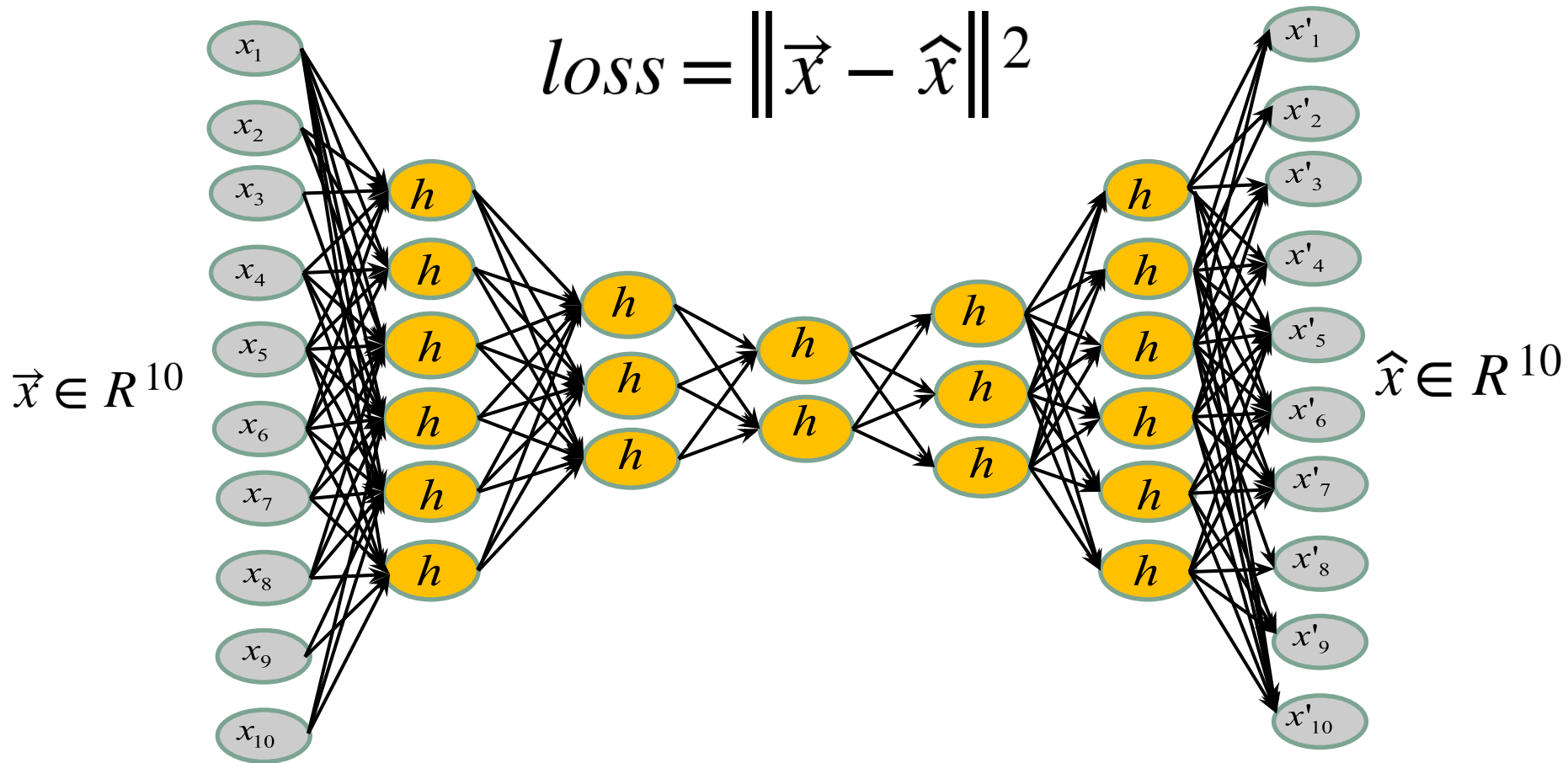
Summary

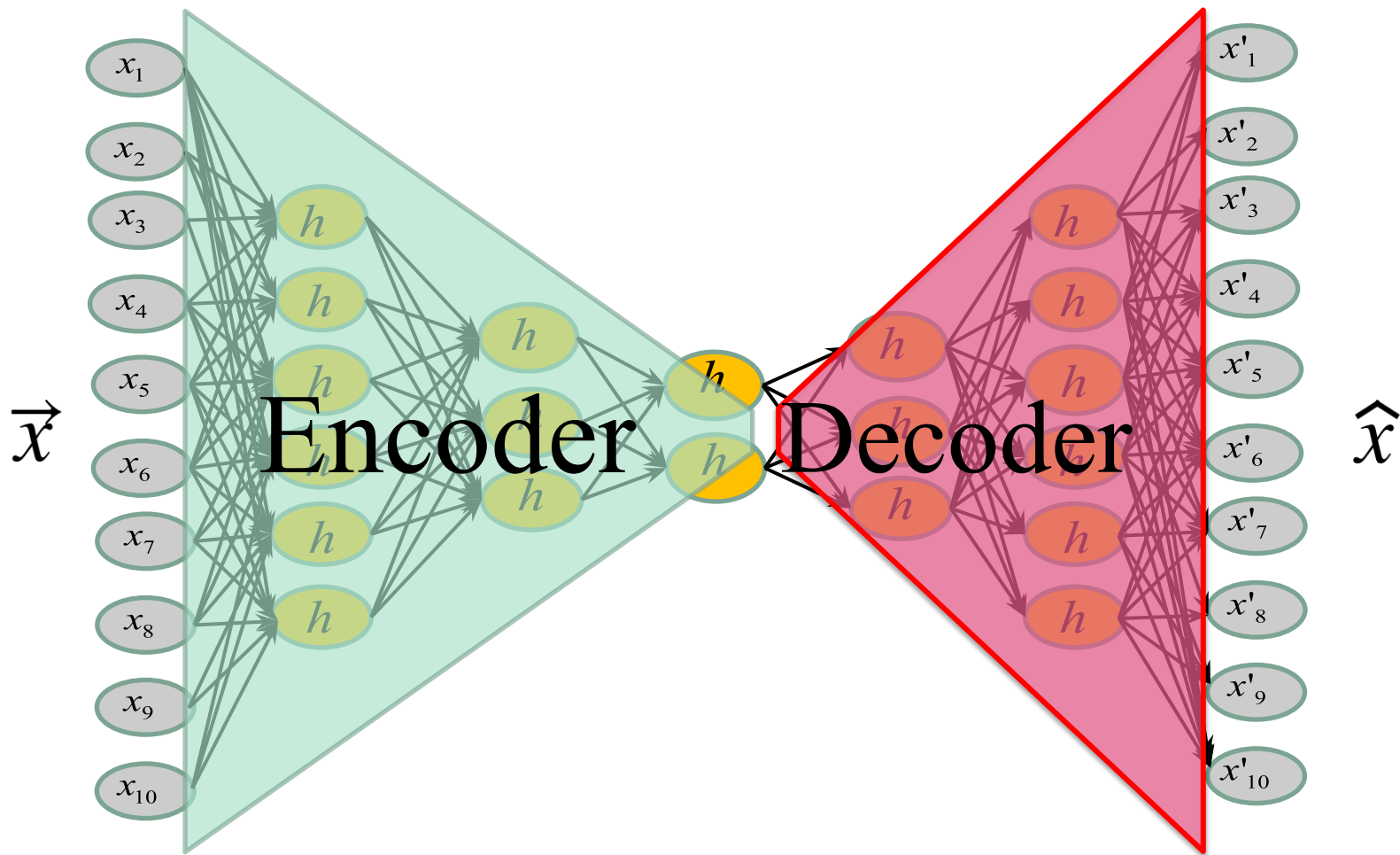
1. What are autoencoders and variational autoencoders?
2. How do they apply to MNIST (grayscale images)?
3. How do they apply to segmentation maps (ACDC cardiac labels)?

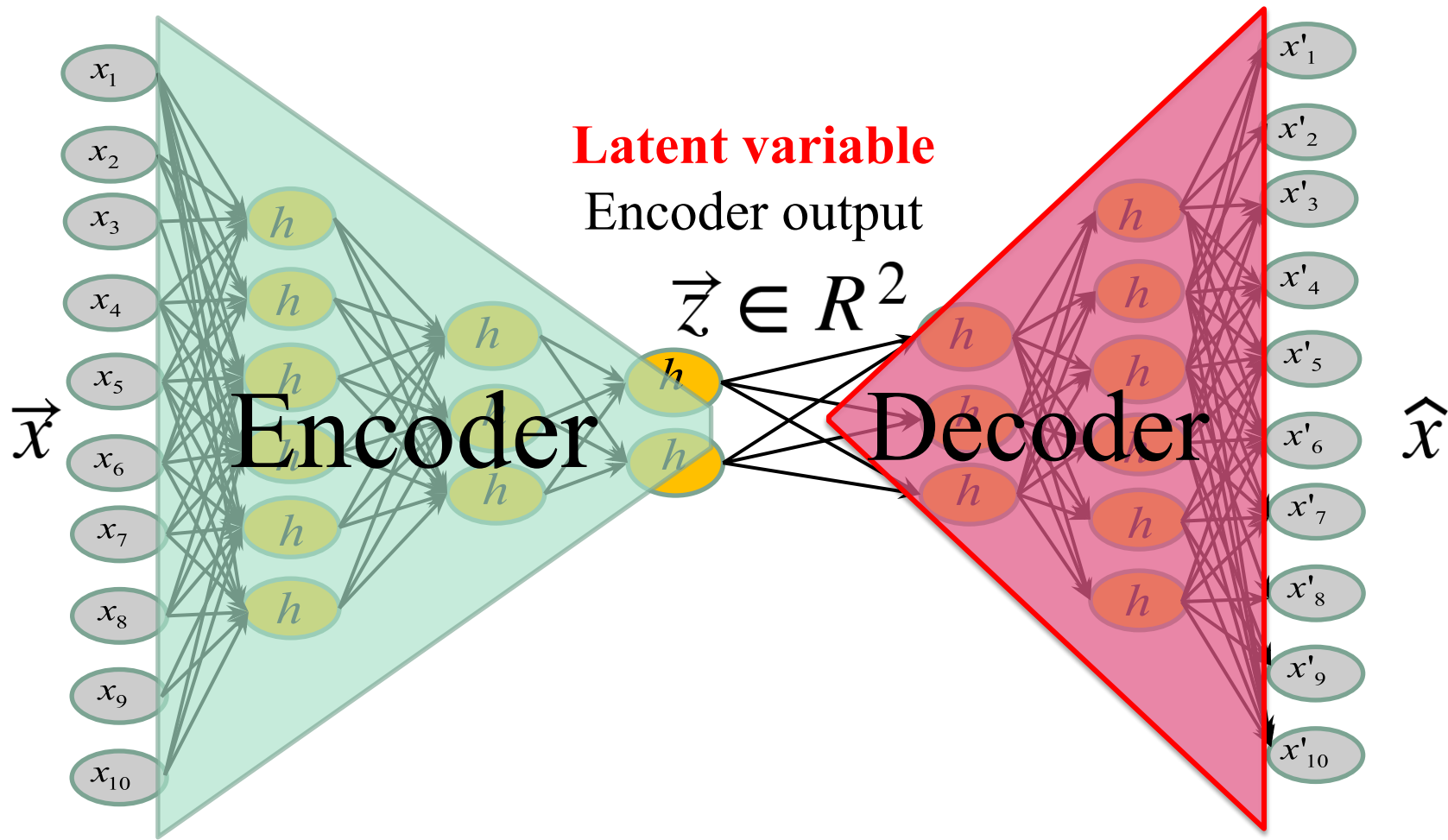
Goal : learn the latent representation of a set of data

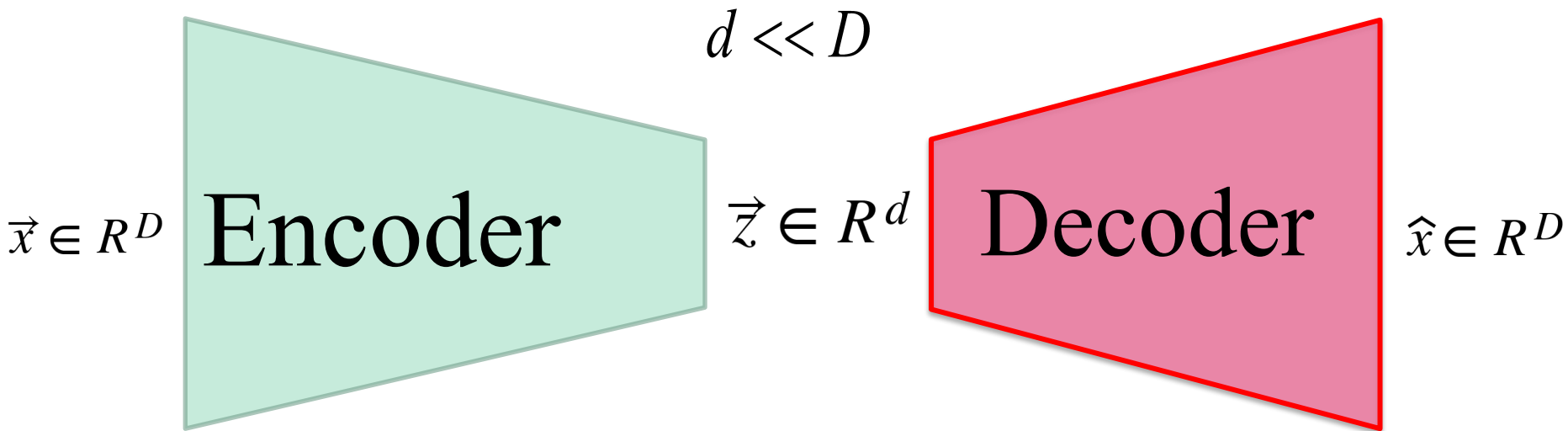
How : by training a Neural Net to output its own ...
input!

Note : if you are familiar with AE and VAE, you may skip the next couple of slides.



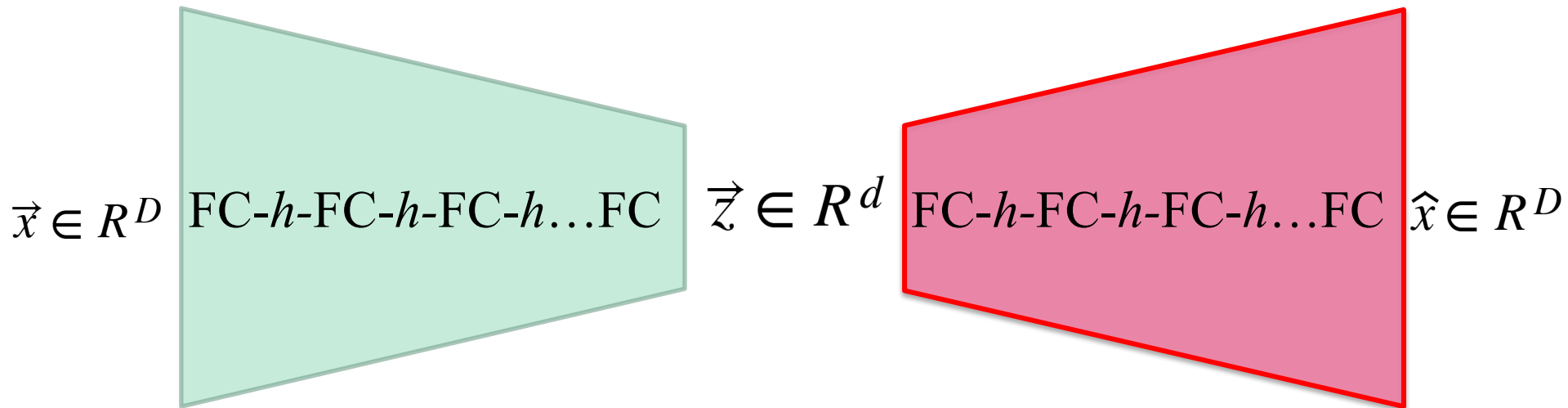






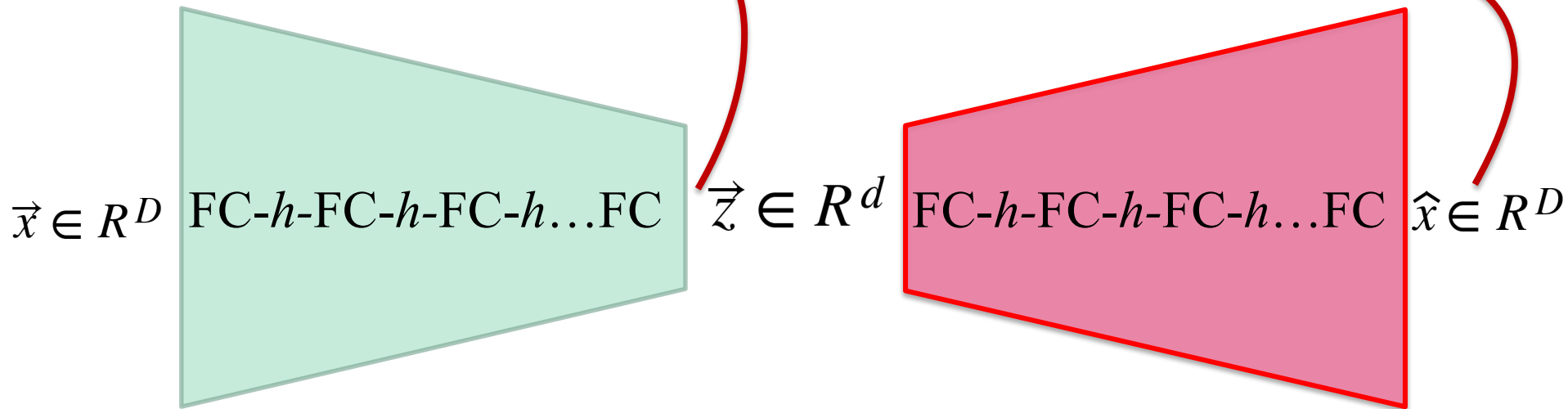
Fully-Connected Layers

h : activation function



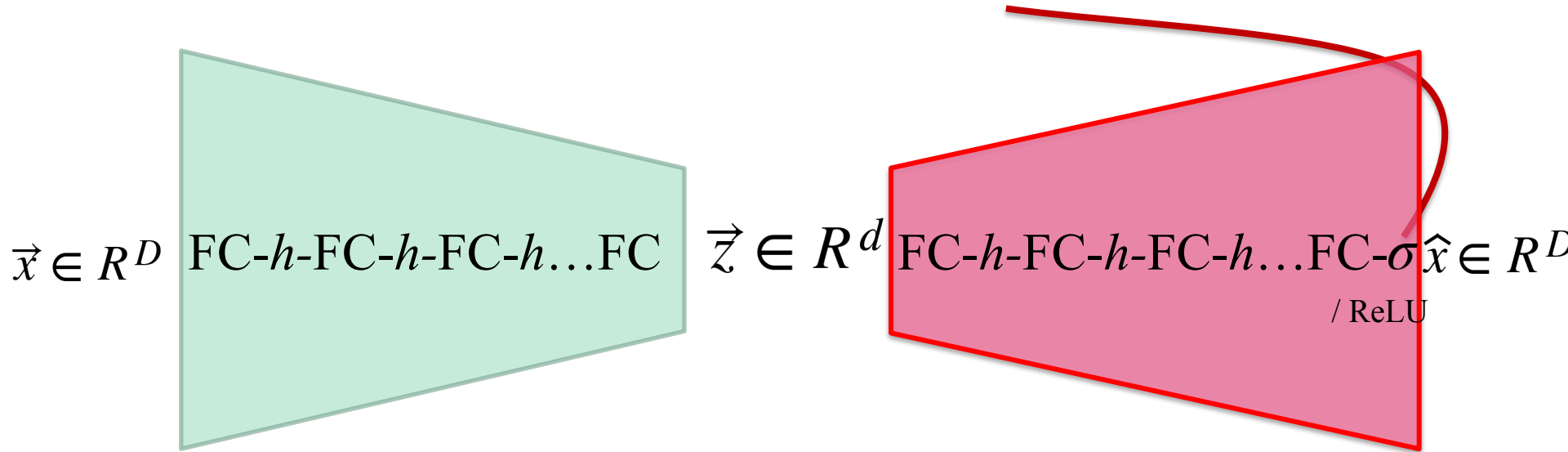
Very often...

No activation function at the output
of the encoder and the decoder



See **why?**

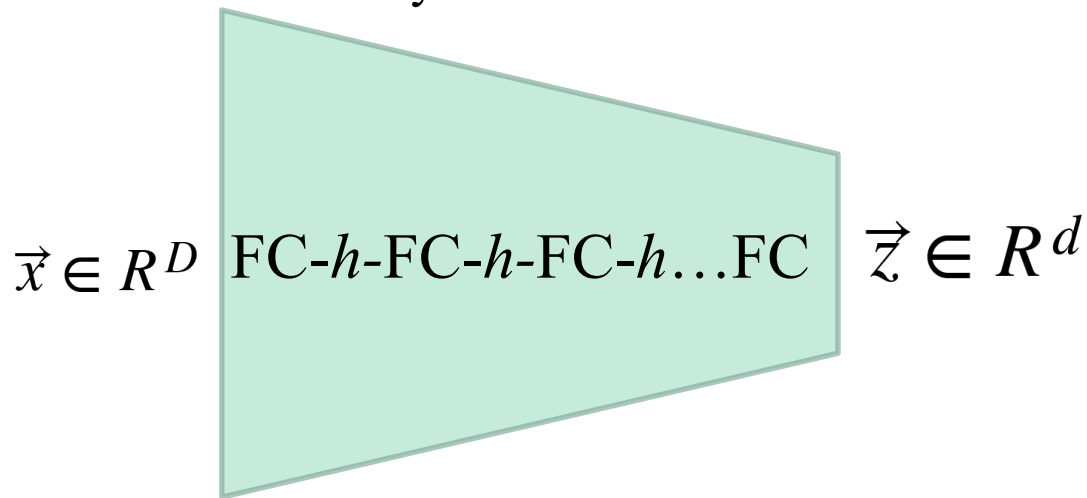
Sometimes **sigmoid** to predict pixel values between 0 and 1 or **ReLU** when the pixel values can be large but never negative.



The number of neurons

Decrease (or stay the same)

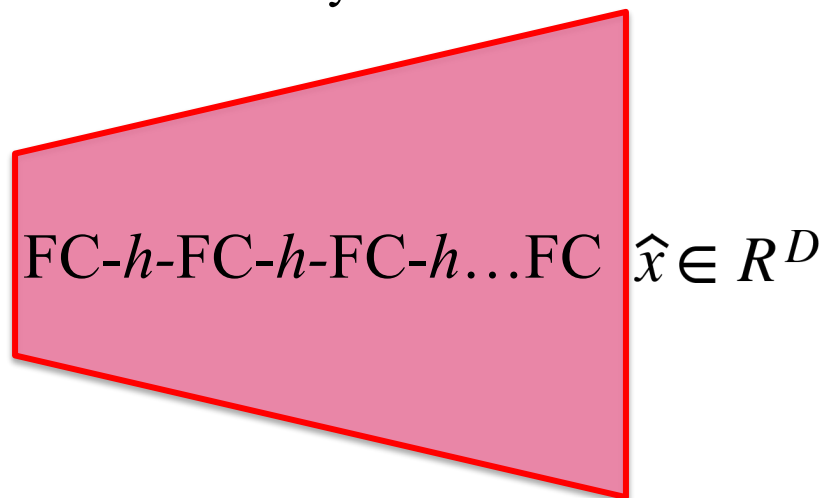
from a layer to another



The number of neurons

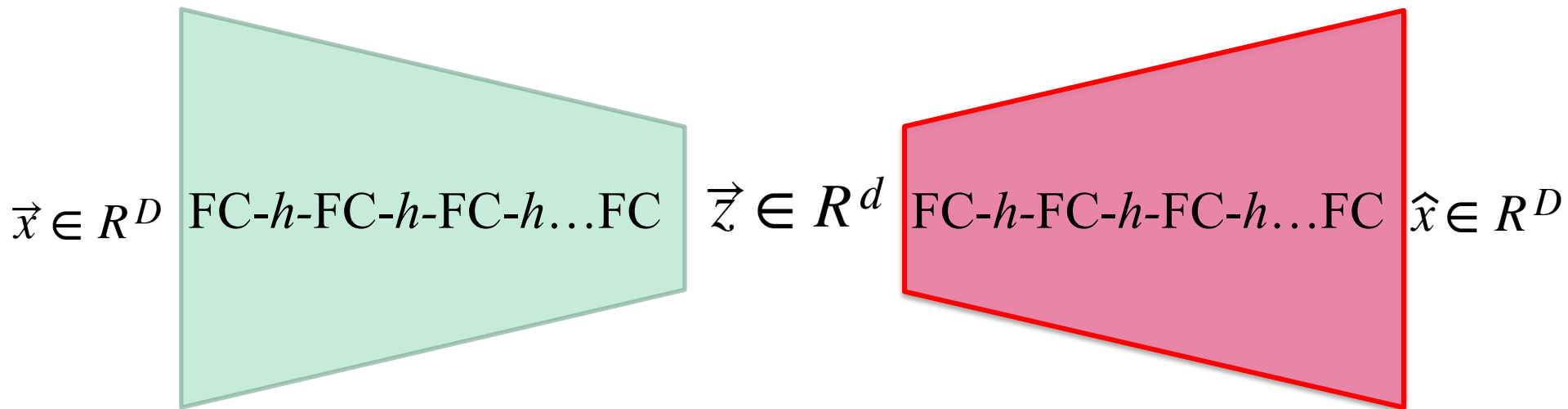
Increase (or stay the same)

from a layer to another

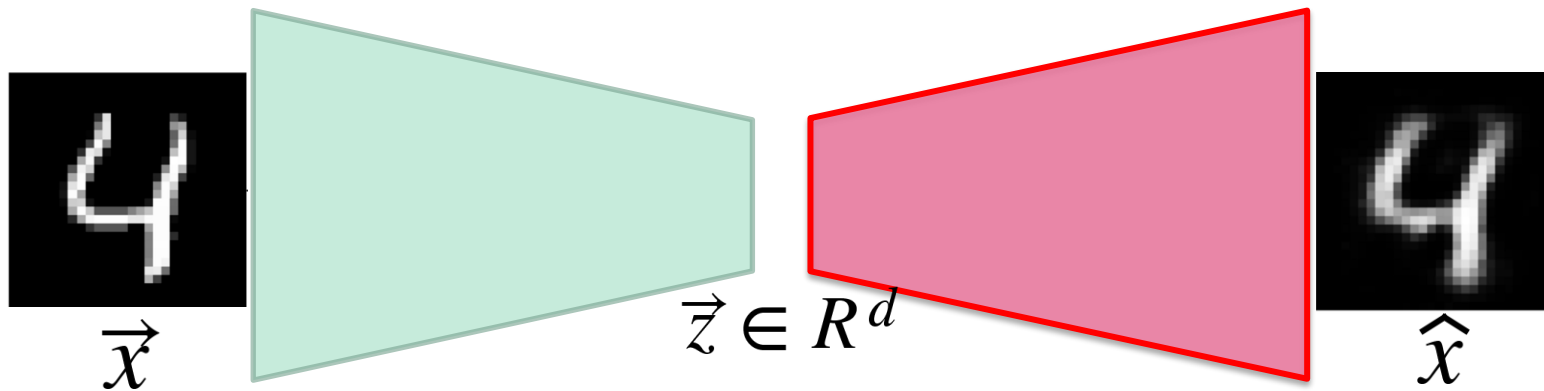


Very often...

The structure of the encoder is the **dual** of that of the decoder



Basic image-based autoencoder



Simple MNIST Autoencoder

```
class autoencoder(nn.Module):  
    def __init__(self):  
        super(autoencoder, self).__init__()  
        self.encoder = nn.Sequential(  
            nn.Linear(28 * 28, 128), nn.ReLU(True),  
            nn.Linear(128, 64), nn.ReLU(True),  
            nn.Linear(64, 12), nn.ReLU(True),  
            nn.Linear(12, 2))  
        self.decoder = nn.Sequential(  
            nn.Linear(2, 12), nn.ReLU(True),  
            nn.Linear(12, 64), nn.ReLU(True),  
            nn.Linear(64, 128), nn.ReLU(True),  
            nn.Linear(128, 28 * 28))  
  
    def forward(self, x):  
        z = self.encoder(x)  
        x_prime = self.decoder(z)  
        return x_prime
```

Latent space 2D



Simple MNIST Autoencoder

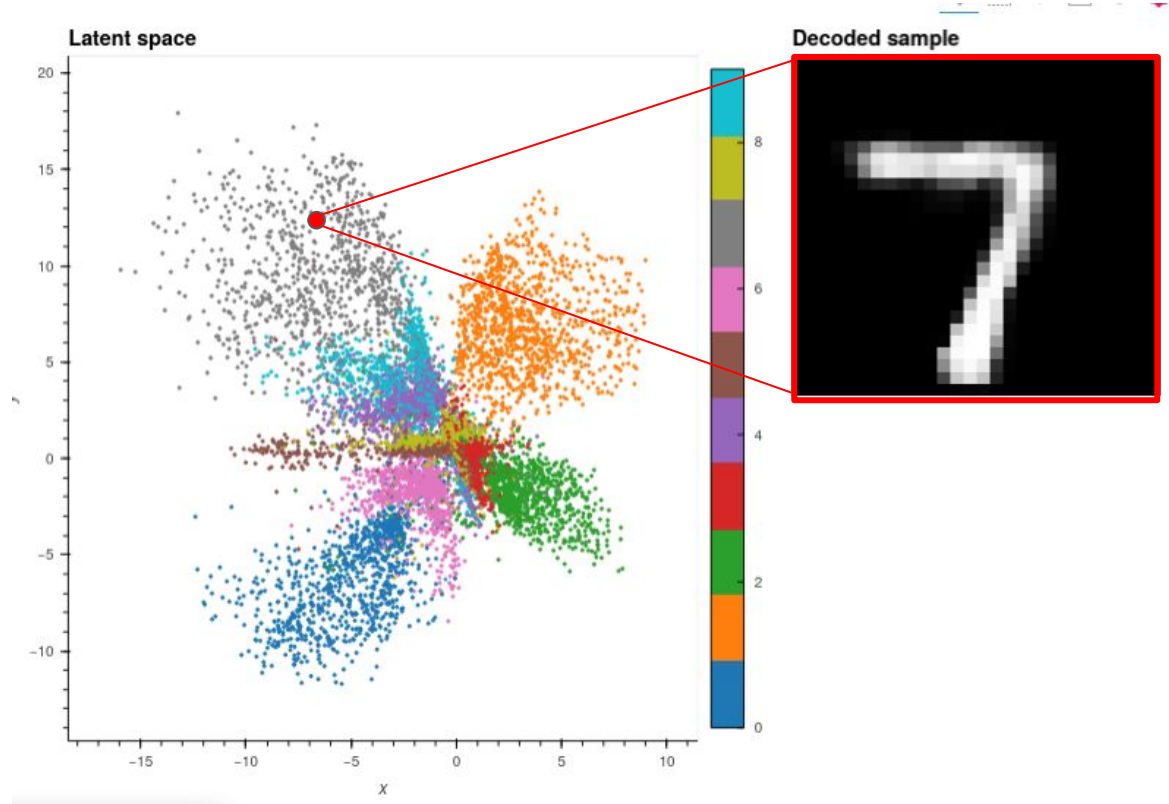
```
class autoencoder(nn.Module):  
    def __init__(self):  
        super(autoencoder, self).__init__()  
        self.encoder = nn.Sequential(  
            nn.Linear(28 * 28, 128), nn.ReLU(True),  
            nn.Linear(128, 64), nn.ReLU(True),  
            nn.Linear(64, 12), nn.ReLU(True),  
            nn.Linear(12, 2))  
        self.decoder = nn.Sequential(  
            nn.Linear(2, 12), nn.ReLU(True),  
            nn.Linear(12, 64), nn.ReLU(True),  
            nn.Linear(64, 128), nn.ReLU(True),  
            nn.Linear(128, 28 * 28))  
  
    def forward(self, x):  
        z = self.encoder(x)  
        x_prime = self.decoder(z)  
        return x_prime
```

symmetry

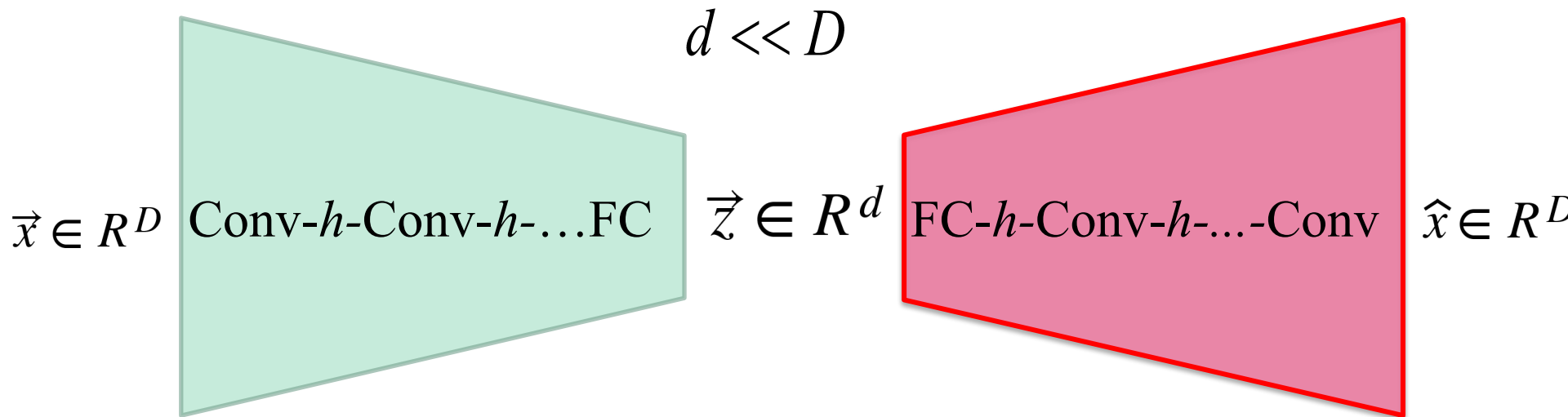


MNIST latent space (for 1000 images)

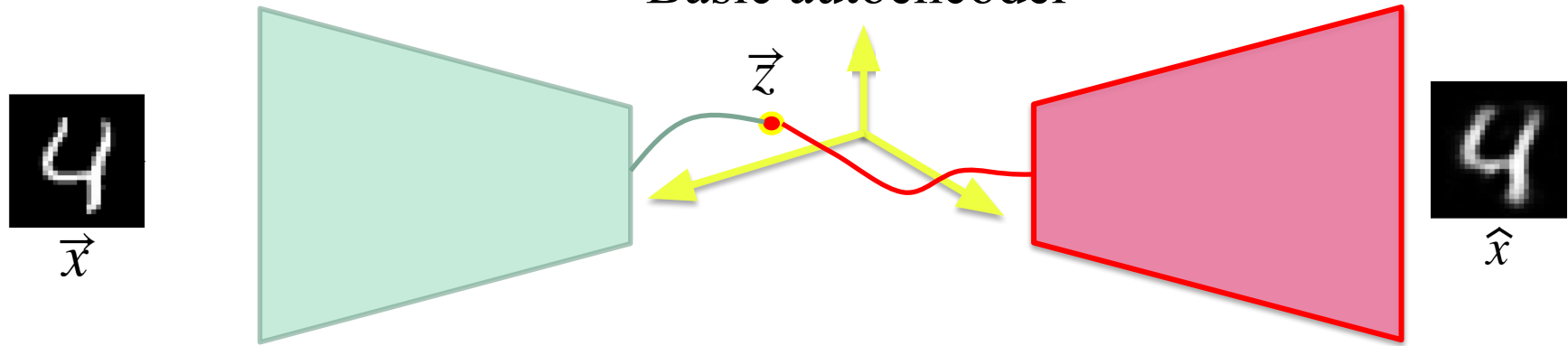
Each 2D point corresponds to an image



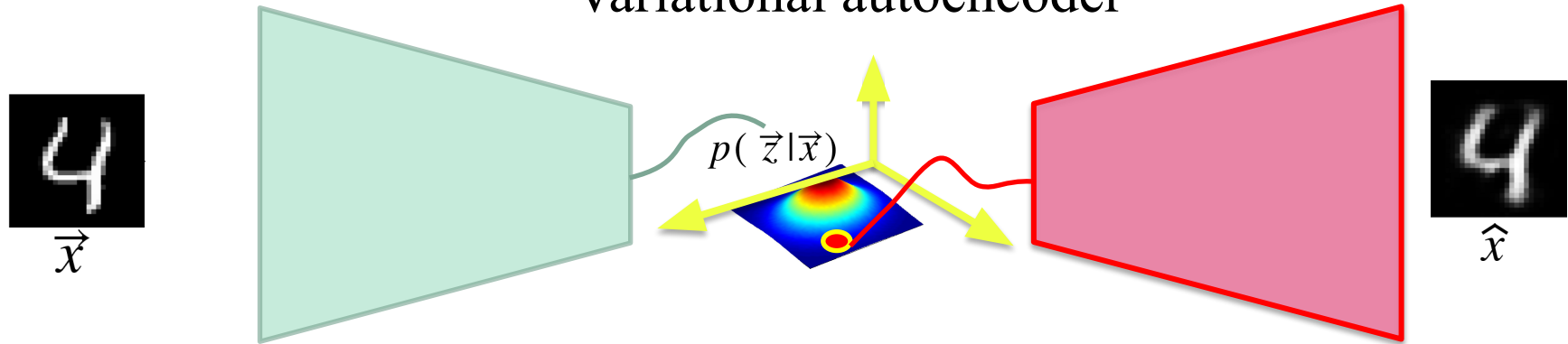
Conv layers



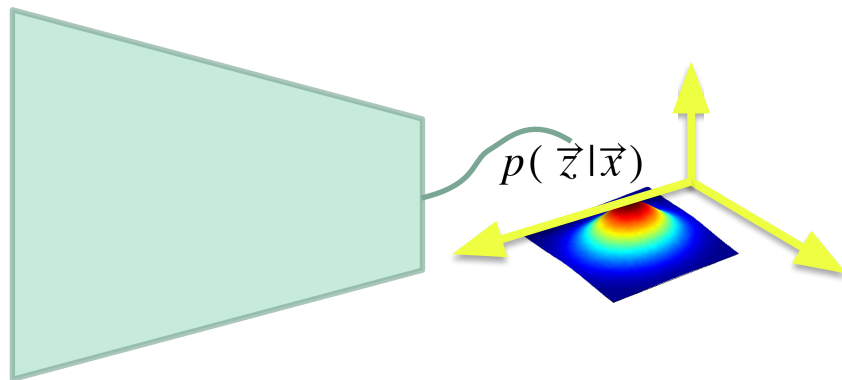
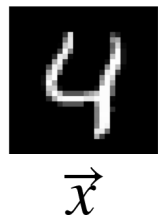
Basic autoencoder



Variational autoencoder

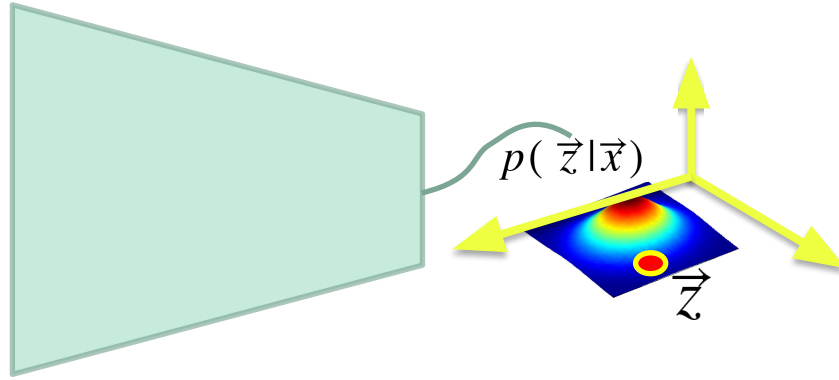
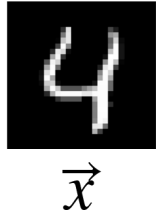


Variational autoencoder



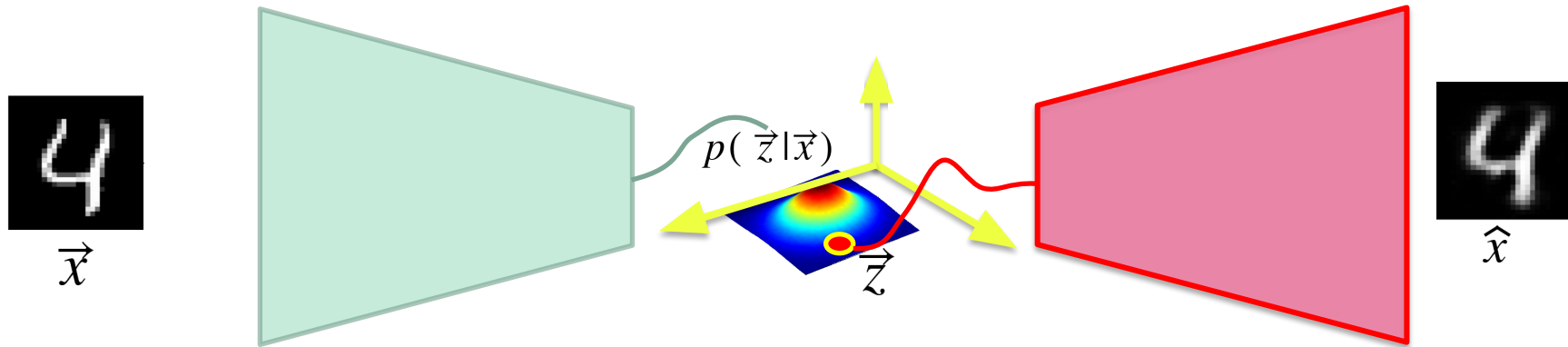
The encoder outputs a **distribution** $p(\vec{z}|\vec{x})$
and not just a **vector** $\vec{z}$

Variational autoencoder



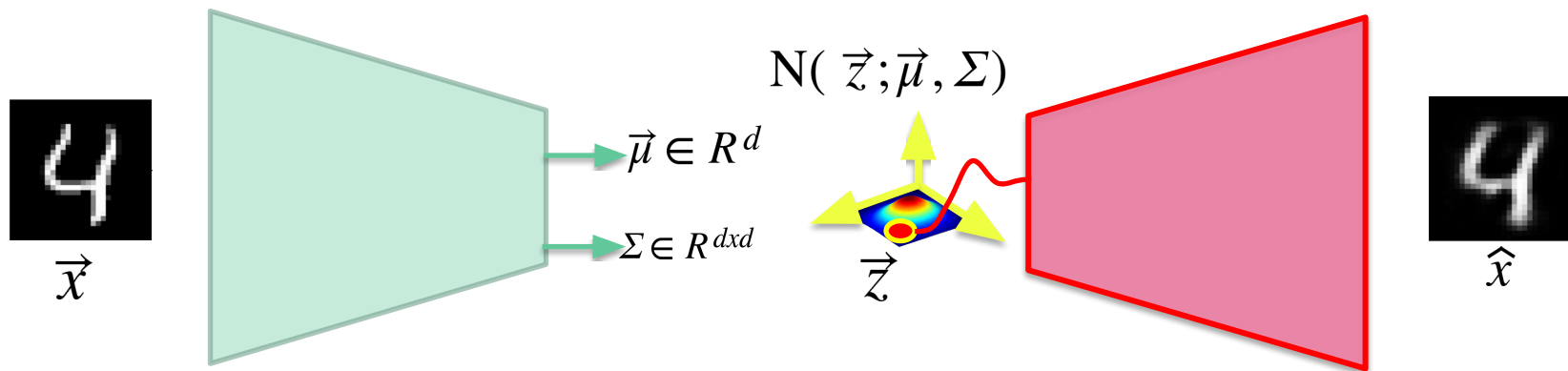
Random sample $\vec{z} \sim P(\vec{z}|\vec{x})$

Variational autoencoder

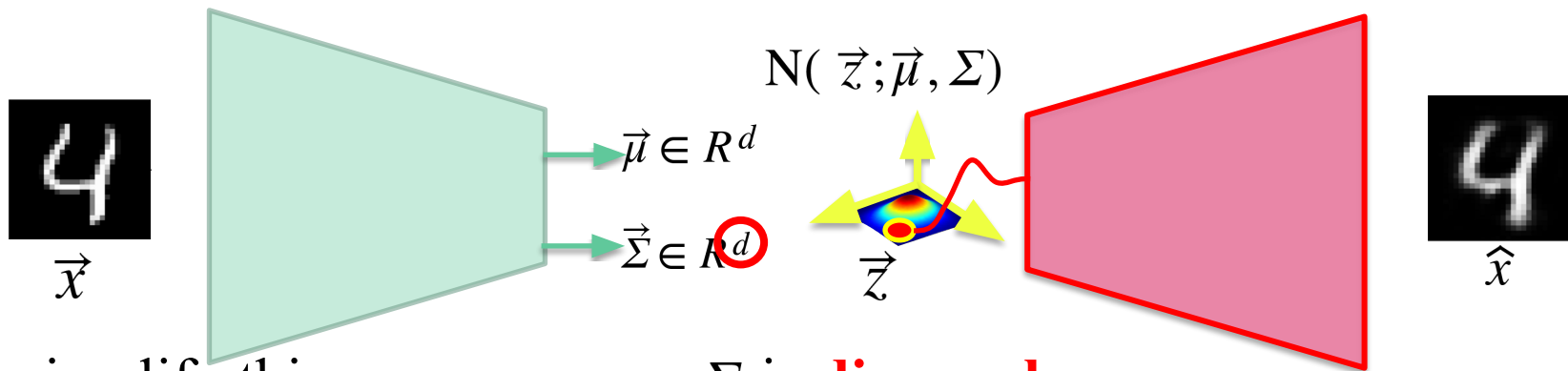


...and rebuild $\hat{x}$

$$p(\vec{z}|\vec{x}) \sim \textit{Gaussian}$$



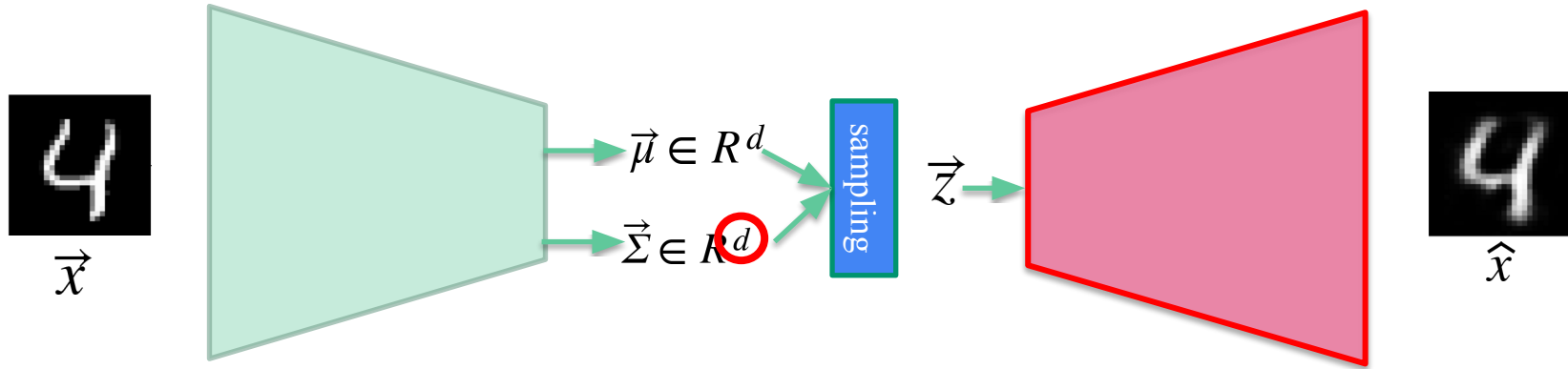
$$p(\vec{z}|\vec{x}) \sim \textit{Gaussian}$$



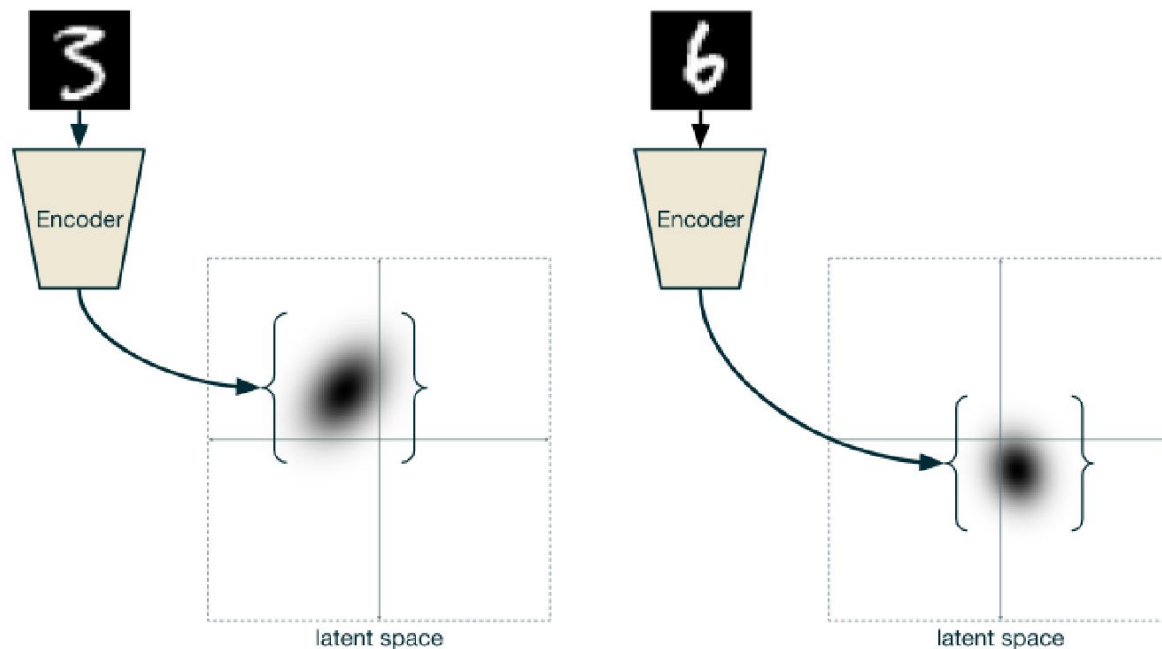
To simplify things, we assume Σ is **diagonal**

$$\Sigma = \begin{pmatrix} \sigma_1^2 & 0 & 0 & 0 \\ 0 & \sigma_1^2 & 0 & 0 \\ 0 & 0 & \ddots & 0 \\ 0 & 0 & 0 & \sigma_d^2 \end{pmatrix}$$

Variational autoencoder

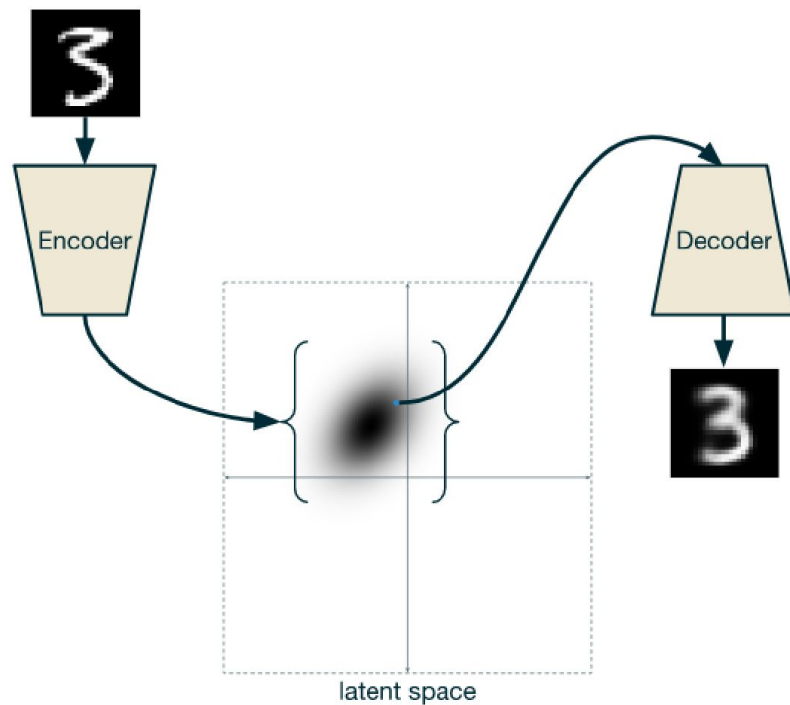


Another way of seeing things...



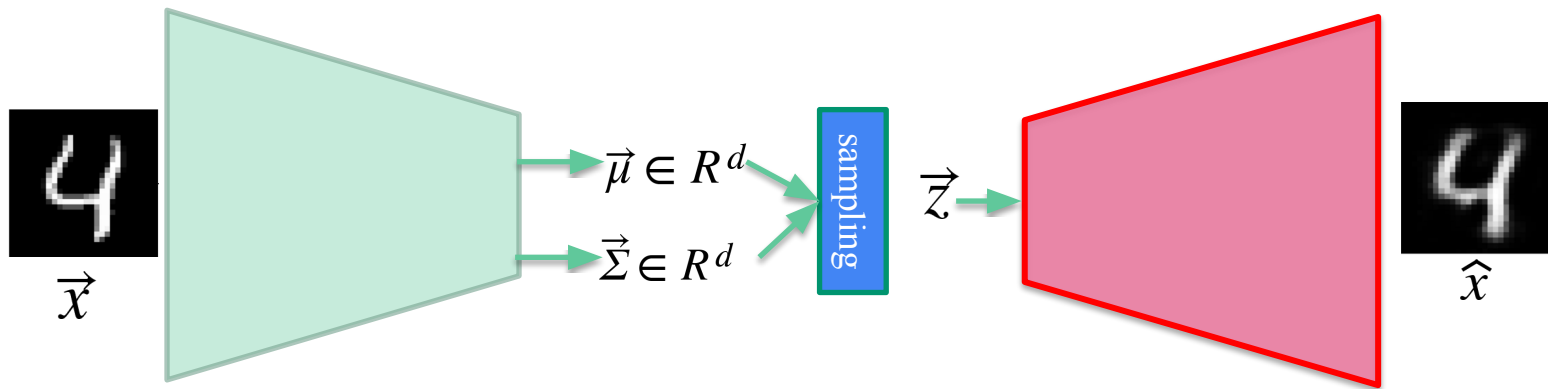
<https://ijdykeman.github.io/ml/2016/12/21/cvae.html>

Another way of seeing things...



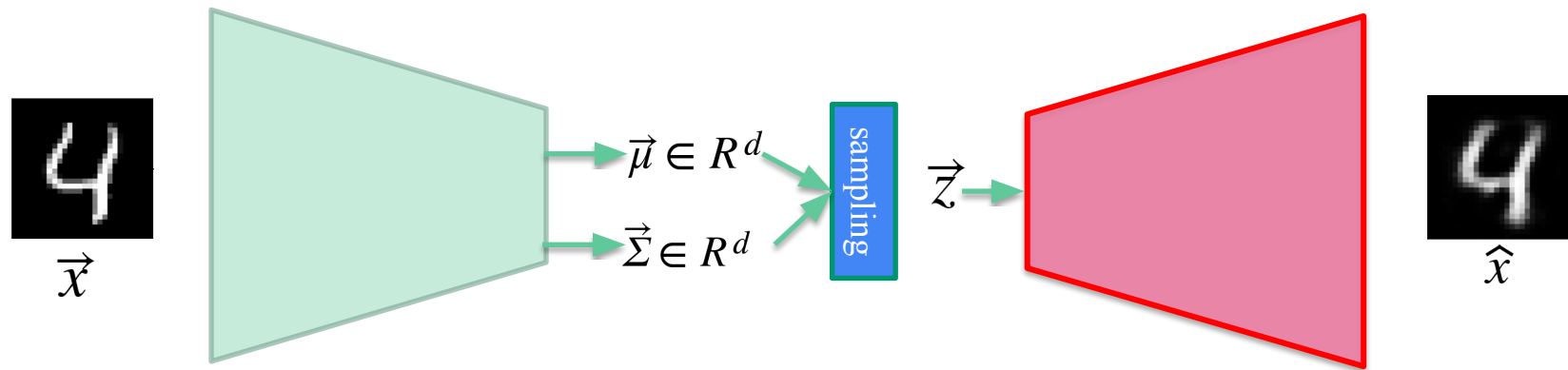
<https://ijdykeman.github.io/ml/2016/12/21/cvae.html>

ELBO loss : Evidence Lower Bound loss



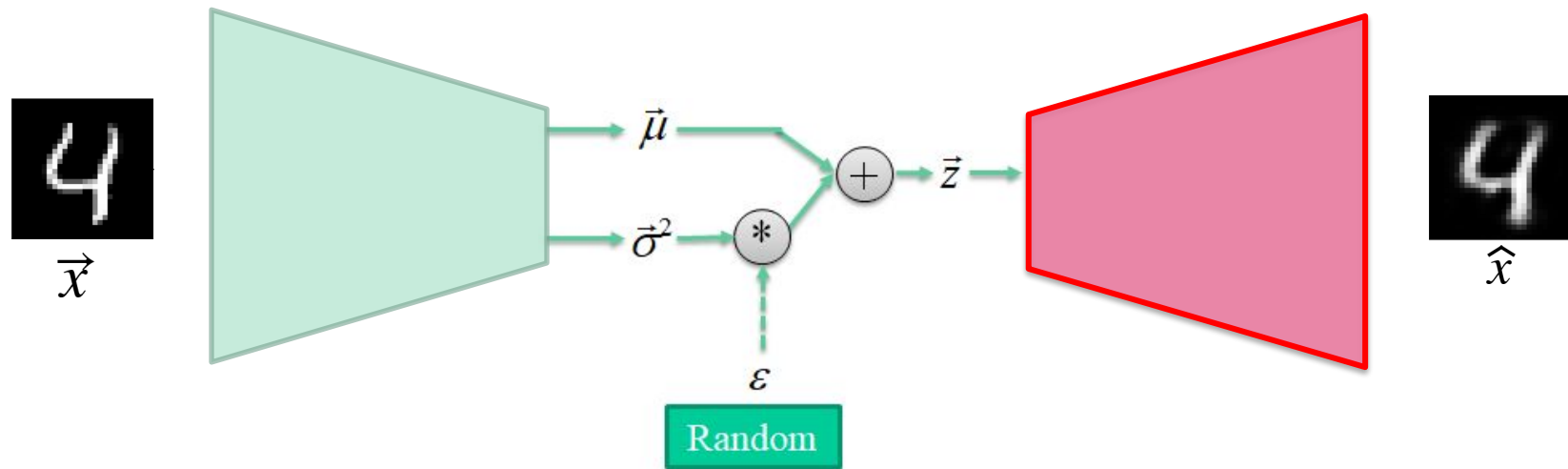
$$Loss = \underbrace{(\vec{x} - \hat{x})^2}_{\text{Loss decoder}} + \underbrace{\lambda KL(N(\vec{z}; \vec{0}, \vec{1}), N(\vec{z}; \vec{\mu}, \vec{\Sigma}))}_{\text{Loss encoder}}$$

Other loss (in case the output is binary)

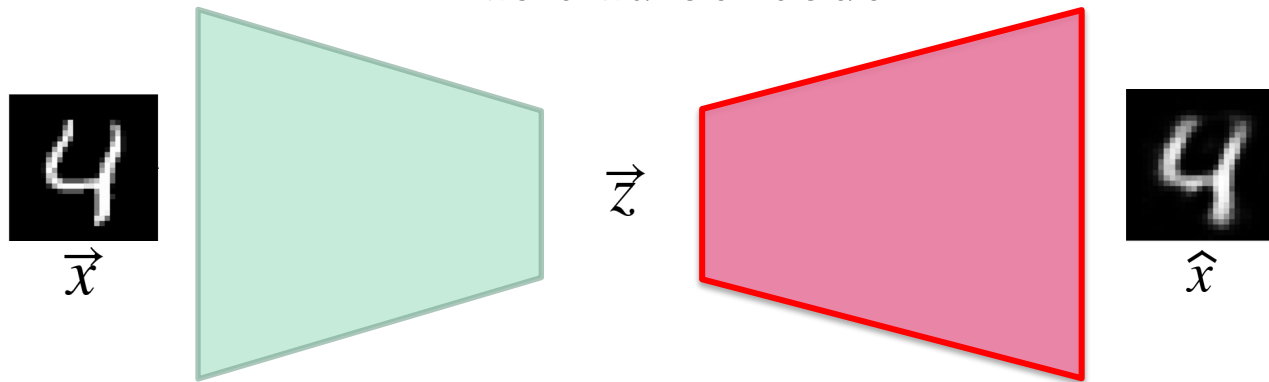


$$\text{Loss} = \underbrace{\text{CrossEntropy}(\vec{x}, \hat{x})}_{\text{Loss decoder}} + \underbrace{\lambda \text{KL}\left(\text{N}(\vec{z}; \vec{0}, \vec{1}), \text{N}(\vec{z}; \vec{\mu}, \vec{\Sigma})\right)}_{\text{Loss encoder}}$$

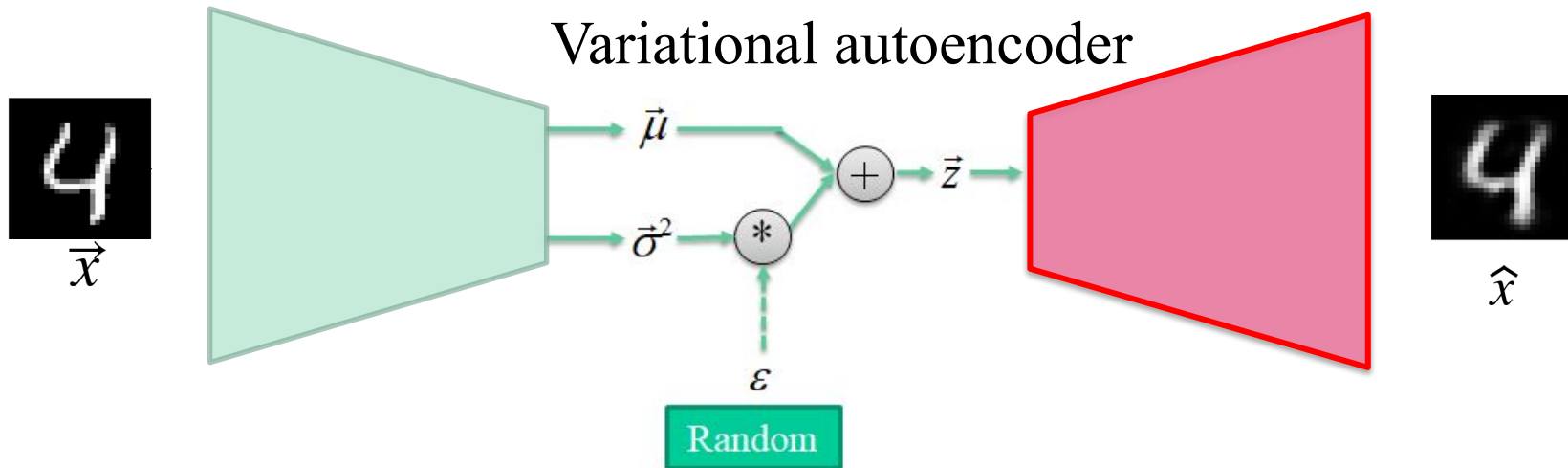
Reparametrization trick instead of sampling



Basic autoencoder

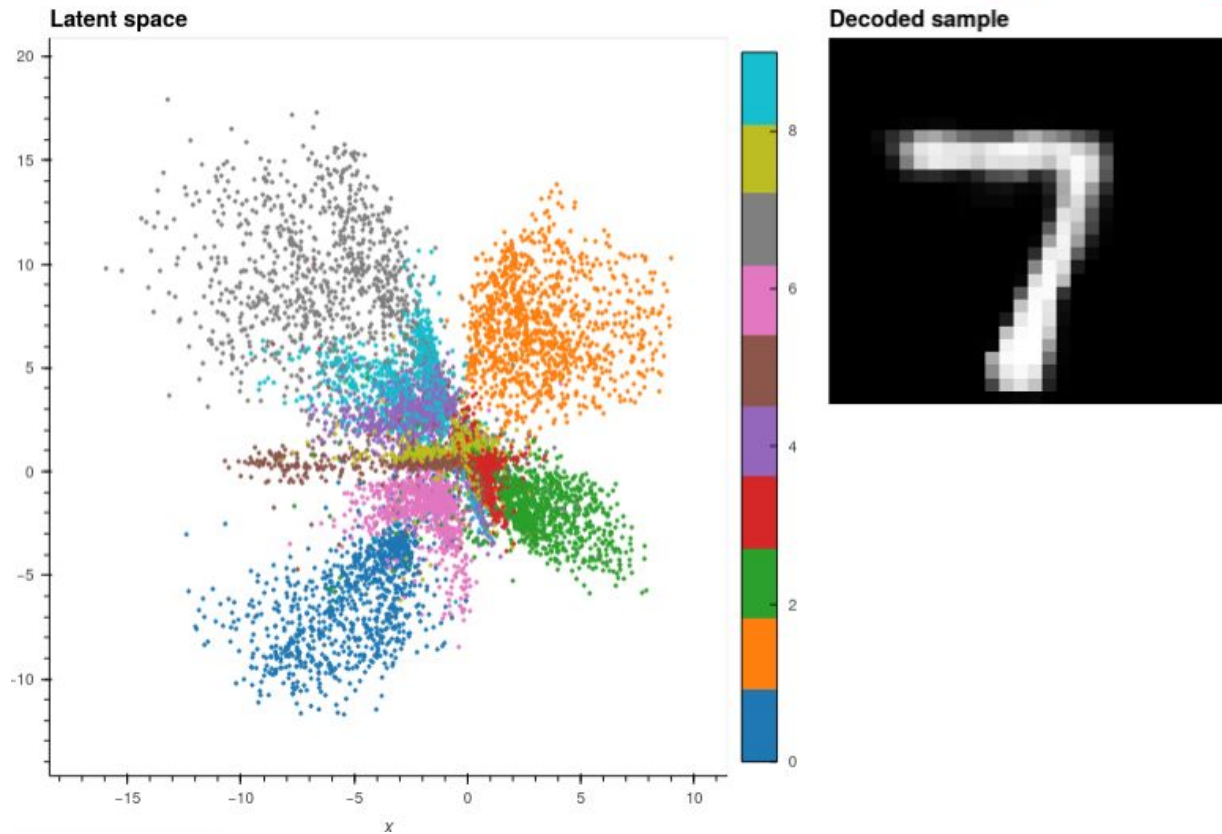


Variational autoencoder



MNIST

Visualize the latent space (autoencoder)



MNIST

Visualize the latent space (variational autoencoder)

